

Lab Test: Revision and Practical for OOP Lab Test 2.

This version includes **objectives**, **topics**, and clear instructions for both **Part A (Debugging)** and **Part B (Programming)** — fully aligned with Chapters 2, 5, and 6.

Lab Test: Revision and Practical (OOP – Java)

1. Objective

The objective of this Lab Test: Revision and Practical is to reinforce students' understanding and application of core Object-Oriented Programming (OOP) concepts using Java. Students will demonstrate their ability to:

- Identify and correct syntax and logic errors in a Java program (**Debugging**).
 - Translate a UML class diagram into a working Java program with correct use of class structures and relationships.
 - Apply foundational OOP principles, including object creation, method definition, and working with `ArrayList`.
-

2. Topics Covered

This lab will focus on **basic and intermediate OOP elements in Java**, particularly:

Included:

- **Chapter 2:** Object and Class
 - Creating and using user-defined classes (String , Scanner , JOptionPane Java-defined class)
 - Instance and static variables
 - Constructors and methods

- **Chapter 5:** ArrayList
 - Creating `ArrayList` of primitive and object types
 - Using `.add()`, `.get()`, `.size()`, `.remove()`
 - Iterating with `for` / `for-each` loop
- **Chapter 6:** Class Relationships
 - **Association** (one class uses another)
 - **Aggregation** (has-a, independent lifecycle)
 - **Composition** (has-a, dependent lifecycle)

✗ Not Covered:

- `enum`
- `package` and `import` custom packages
- Java arrays (`int[]`, `String[]`, etc.) - used but not the question type focus . Example in main header.
- `Vector` (to be covered in Final Exam)

3. Structure of the Lab Test

The test is divided into two main parts:

Part A: Debugging (30 - 40 Marks)

You are given a faulty `.java` file containing both **syntax errors** (e.g., missing semicolons, wrong data types) and **logic errors** (e.g., incorrect method calls, faulty calculations, missing object references).

- Based on the previous **Lab Test 2 (Test 2)** pattern.
- The buggy code will typically involve class instantiation, method calling, `ArrayList`, and class relationships (Chapter 2 to Chapter 6 elements).
- Your task is to:
 - Fix all errors.
 - Ensure the corrected code compiles and runs successfully.
 - Produce the expected output shown in a provided figure (e.g., “Figure A-1”).

 Note to Students:

Students are encouraged to revise how you solved it — don’t just copy the answer . Learning process must be performed (No shortcut) .

 AI Usage Warning:

If you use AI tools (e.g., ChatGPT, Copilot) to get a direct answer without understanding the steps, **you must re-attempt the process on your own**. Copying solutions without comprehension will not help you in the Test/Final Exam and may impact your marks.

 Part B: Programming – Based on UML (60-70 Marks)

You will be given a **UML class diagram** involving class relationships. Your task is to:

1. Understand and interpret the class relationships (association, aggregation, composition).
2. Implement the classes in Java with:
 - Correct attributes and visibility
 - Constructors
 - Getters and setters (as necessary)
 - Methods (as described)
3. Write a `main` method to test the classes (object creation, method calls, output)

 Example Themes:

- Staff–Address (Aggregation)
- Staff-id (Composition)
- Staff - Course (Association)

 Deliverable:

- `.java` file(s) with proper structure and output
 - Include comments to explain key parts (optional but encouraged)
-

 Tips for Success

- Read the question thoroughly before starting.
- Download the provided folder to your PC. The internet connection will be turned off after the download is complete.
- For Part A: Run your code often — compile early, fix errors step by step.
- For Part B: Break the UML into class-by-class implementations.
- Use `ArrayList<ClassName>` correctly with object storage.
- Make sure constructors and methods match the requirements.
- Don't forget : after successful compile, test with the sample input and display output (as given in sample output).